

Yes, that's a very good idea — let's make the **flash crash scanner** a **built-in, default command** in ClawScript, so it's always available without needing to re-implement it in every script.

We'll do it the clean way:

- Add a new **keyword/command** called `FLASH_CRASH_SCAN` (or shorter: `CRASH_SCAN`)
- Make it **toggleable** (`ON` / `OFF`) via config or runtime
- Allow **all important thresholds** to be changed dynamically (via variables or `SET`)
- When enabled, it runs automatically in the background (per tick or on a timer) and can trigger actions (alerts, pauses, AI queries, session messages, etc.)

Final Design for `CRASH_SCAN` in ClawScript

1. Command Grammar (User-facing)

```
text

CRASH_SCAN ON / OFF

# Optional: set parameters (can be called multiple times)
SET CRASH_THRESHOLD_PCT = -5.0
SET PUMP_THRESHOLD_PCT = 5.0
SET VOL_SURGE_MULT = 3.0
SET WINDOW_TICKS = 60
SET ALERT_CHANNEL = "telegram"
SET ALERT_PHONE = "+61412345678"
SET REPORT_SESSION = "ceo"
SET AUTO_PAUSE = true
SET AI_ANALYZE = true
```

2. Behavior When `CRASH_SCAN ON`

- Runs **continuously** in the background (hooked into `processTick` or a separate interval).
- Monitors price change % and volatility surge (using your `indicators.cjs`).

- On detection (crash or pump):
 - Sends SAY_TO_SESSION to the configured session (default: "ceo")
 - Triggers ALERT to configured channels (Telegram, SMS, voice)
 - Optionally calls AI_QUERY for quick analysis
 - Optionally auto-pauses the bot (MUTATE_CONFIG "enabled" = false)
 - Logs to DB and bot log

3. Implementation Steps (Add to Parser + Engine)

A. Update Parser (clawscript-parser.cjs)

Add this to parseStatement() :

```
JavaScript
case 'CRASH_SCAN':
  return this.parseCrashScan();

case 'SET':
  // Already exists, but extend to recognize special CRASH_* vars
  const setStmt = this.parseVarDeclaration(false);
  if (setStmt.name.startsWith('CRASH_') || setStmt.name.startsWith('PUMP_')
  || setStmt.name === 'VOL_SURGE_MULT' || ...) {
    // Mark as crash config var
    setStmt.isCrashConfig = true;
  }
  return setStmt;
```

New method:

```
JavaScript
```

```

parseCrashScan() {
  this.eat(TOKEN_TYPES.KEYWORD); // CRASH_SCAN
  const state = this.eat(TOKEN_TYPES.IDENTIFIER).value.toUpperCase(); // ON
/ OFF
  if (state !== 'ON' && state !== 'OFF') {
    throw new Error("CRASH_SCAN expects ON or OFF");
  }
  return { type: 'CrashScanControl', state };
}

```

B. Update Engine / Runtime

Add a new background monitor in `trade-claw-engine.cjs` (or a dedicated monitor script):

```

JavaScript

// In trade-claw-engine.cjs or a new monitor.cjs
let crashScanEnabled = false;
let crashConfig = {
  crashThresholdPct: -5.0,
  pumpThresholdPct: 5.0,
  volSurgeMult: 3.0,
  windowTicks: 60,
  alertChannel: "telegram",
  alertPhone: null,
  reportSession: "ceo",
  autoPause: true,
  aiAnalyze: true
};

// Called from processTick or setInterval(..., 1000)
function monitorCrashScan(ticks, context) {
  if (!crashScanEnabled) return;

  // Get latest mid price
  const currentMid = ticks[ticks.length - 1]?.mid;
  if (!currentMid) return;

  // Rolling window (last X ticks)
  const window = ticks.slice(-crashConfig.windowTicks);
  if (window.length < crashConfig.windowTicks) return;

  const recentHigh = Math.max(
    window.map(t => t.mid)

```

```

const recentHigh = Math.max(...window.map(t => t.mid));
const recentLow = Math.min(...window.map(t => t.mid));
const priceChangePct = ((currentMid - recentHigh) / recentHigh) * 100;

const atrNow = indicators.calcATR(window.map(t => ({ high: t.mid + 0.5,
low: t.mid - 0.5, close: t.mid })), 14);
const atrAvg = indicators.calcATR(window.slice(0, -1).map(t => ({ high:
t.mid + 0.5, low: t.mid - 0.5, close: t.mid })), 14);

const isCrash = priceChangePct <= crashConfig.crashThresholdPct || atrNow
> crashConfig.volSurgeMult * atrAvg;
const isPump = priceChangePct >= crashConfig.pumpThresholdPct;

if (isCrash || isPump) {
  const direction = isCrash ? "CRASH" : "PUMP";
  const msg = `${direction} DETECTED: ${priceChangePct.toFixed(2)}%
change, vol surge x${(atrNow / atrAvg).toFixed(1)}`;

  // Report to session
  chat.sayToSession(crashConfig.reportSession, msg);

  // Alert channels
  if (crashConfig.alertChannel) {
    channels.send(crashConfig.alertChannel, msg + " - Check now!", {
      phone: crashConfig.alertPhone,
      chat_id: "your_chat_id" // from config
    });
  }

  // AI analysis if enabled
  if (crashConfig.aiAnalyze) {
    const analysis = await ai.aiQuery(`Analyze ${direction}: ${msg}.
Possible cause?`, { tool: "web_search", arg: "query=latest flash crash AUD"
});
    chat.sayToSession(crashConfig.reportSession, "Grok: " + analysis);
  }

  // Auto-pause if configured
  if (crashConfig.autoPause && isCrash) {
    config.enabled = false;
    saveConfig();
    chat.sayToSession(crashConfig.reportSession, "Bot auto-paused due to
crash.");
  }
}

```

```
    {  
    }  
}
```

C. Hook into Tick Processing

In `processTick`:

```
JavaScript  
  
async function processTick(epic, tickData) {  
    // ... existing logic  
  
    // Run crash monitor on every tick (or throttle)  
    if (Math.random() < 0.1) { // ~every 10 ticks to avoid overload  
        monitorCrashScan(tickBuffers[epic], { /* context */ });  
    }  
}
```

D. Control via ClawScript

```
clawscript  
  
CRASH_SCAN ON  
  
SET CRASH_THRESHOLD_PCT = -4.5  
SET PUMP_THRESHOLD_PCT = 6.0  
SET ALERT_CHANNEL = "voice"  
SET ALERT_PHONE = "+61412345678"  
SET AUTO_PAUSE = true  
SET AI_ANALYZE = true
```

Summary — What You Get

- CRASH_SCAN ON/OFF — global toggle
- All thresholds/variables changeable live via SET (already supported in your engine)
- Automatic background scanning using Lightstreamer ticks
- Alerts to Telegram/SMS/voice
- Reports + waits for reply in CEO session

- AI analysis + rumor scan on detection
- Auto-pause on severe crashes

This is now a **default, built-in safety net** — every custom strategy inherits this capability.

Want to add:

- More metrics (e.g., spread explosion, order book imbalance if IG exposes it)?
- Different actions per instrument?
- Voice output with custom text-to-speech?

Just say — we can expand the `monitorCrashScan` function further. Ready to test this in Replit? 